

Shortest Paths on DAGS

Shortest Paths on Directed Acyclic Graphs (DAGs)

Finding shortest paths in Directed Acyclic Graphs (DAGs) can be computationally easier than in general graphs, and the algorithm even works correctly with negative edge weights (which Dijkstra's algorithm cannot handle).

Challenge

The primary challenge in general shortest path problems is dealing with cycles, especially negative cycles, which can lead to infinitely short paths. DAGs, by definition, have no cycles, simplifying the problem.

Algorithm Idea

The algorithm for finding shortest paths in a DAG leverages its acyclic nature by processing vertices in a specific order: **topological order**.

1. **Topological Sort:** First, perform a topological sort of the DAG. This produces a linear ordering of its vertices such that for every directed edge (u, v) , u comes before v in the ordering.
2. **Initialize Distances:** Initialize $s.d = 0$ for the source vertex s , and $v.d = \infty$ for all other vertices v .
3. **Relax Edges in Order:** Iterate through the vertices in their topological order. For each vertex u in this order, relax all its outgoing edges (u, v) .

This approach ensures that when an edge (u, v) is relaxed, the shortest path to u has already been finalized, as u appears before v in the topological sort.

Works with Negative Edge Weights

Unlike Dijkstra's algorithm, this DAG shortest path algorithm correctly handles negative edge weights because there are no cycles. Negative cycles are the only reason negative weights pose a problem for shortest path algorithms (as a path could infinitely decrease its weight by traversing a negative cycle).

Proof of Correctness (Lemma)

Lemma: The algorithm computes shortest paths from a source s to all other vertices in a DAG.

Proof Intuition: Consider the vertices in topological order. When we process a vertex u , all its predecessors have already been processed. This means that any path to u from s consists of vertices that have already been processed (and their distances finalized) or s itself. When u is processed, all its outgoing edges (u, v) are relaxed. Since all paths to u have been considered and $u.d$ is correct, relaxing (u, v) will correctly update $v.d$ if a shorter path through u exists. Because of the topological order, we are guaranteed to process vertices in an order that

respects dependencies, ensuring that when we relax an edge (u, v) , $u.d$ already holds the shortest path distance from s to u . Thus, $v.d$ will eventually converge to its shortest path distance.

Implementation

1. Compute a topological sort of the graph G .
2. Initialize distance estimates: $s.d = 0$, $v.d = \infty$ for $v \neq s$.
3. Iterate through the vertices u in topological order:
 - For each outgoing edge (u, v) :
 - **RELAX**(u, v) (same relaxation step as in Dijkstra's, but without a priority queue).

Total Time Complexity

The total time complexity is dominated by: - Topological sort: $O(|V| + |E|)$
- Initializing distances: $O(|V|)$ - Iterating through vertices and relaxing edges: Each edge is relaxed exactly once. $O(|V| + |E|)$

Therefore, the total time complexity is $O(|V| + |E|)$. This is highly efficient, especially for sparse graphs.

Related

- [[Graphs]]
- [[Shortest Paths]]
- [[Topological Sort]]